

Adaptive Fusion of Multi-View for Graph Contrastive Recommendation

Mengduo Yang*
Yi Yuan*
Jie Zhou
yangmd2118@mails.jlu.edu.cn
icecens@zju.edu.cn
22251247@zju.edu.cn
Zhejiang University
School of Software Technology
Ningbo, Zhejiang, China

Meng Xi†
Xiaohua Pan
ximeng@zju.edu.cn
panxh@zju-bj.com
Zhejiang University
School of Software Technology
Ningbo, Zhejiang, China
Binjiang Institute of Zhejiang
University
Hangzhou, Zhejiang, China

Ying Li
Yangyang Wu
cnliying@zju.edu.cn
zjuwuyy@zju.edu.cn
Zhejiang University
School of Software Technology
Ningbo, Zhejiang, China
Binjiang Institute of Zhejiang
University
Hangzhou, Zhejiang, China

Jinshan Zhang
zhangjinshan@zju.edu.cn
Zhejiang University
School of Software Technology
Ningbo, Zhejiang, China

Jianwei Yin
zjuyjw@zju.edu.cn
Zhejiang University
School of Software Technology
Ningbo, Zhejiang, China
Binjiang Institute of Zhejiang
University
Hangzhou, Zhejiang, China

ABSTRACT

Recommendation is a key mechanism for modern users to access items of their interests from massive entities and information. Recently, graph contrastive learning (GCL) has demonstrated satisfactory results on recommendation, due to its ability to enhance representation by integrating graph neural networks (GNNs) with contrastive learning. However, those methods often generate contrastive views by performing random perturbation on edges or embeddings, which is likely to bring noise in representation learning. Besides, in all these methods, the degree of user preference on items is omitted during the representation learning process, which may cause incomplete user/item modeling. To address these limitations, we propose the Adaptive Fusion of Multi-View Graph Contrastive Recommendation (AMGCR) model. Specifically, to generate the informative and less noisy views for better contrastive learning, we design four view generators to learn the edge weights focusing on weight adjustment, feature transformation, neighbor aggregation, and attention mechanism, respectively. Then, we employ an adaptive multi-view fusion module to combine different views from both the view-shared and the view-specific levels. Moreover, to make

the model capable of capturing preference information during the learning process, we further adopt a preference refinement strategy on the fused contrastive view. Experimental results on three real-world datasets demonstrate that AMGCR consistently outperforms the state-of-the-art methods, with average improvements of over 10% in terms of Recall and NDCG. Our code is available on <https://github.com/Du-danger/AMGCR>.

CCS CONCEPTS

• Information systems → Recommender systems.

KEYWORDS

Collaborative Filtering, Recommendation, Graph Neural Networks, Contrastive Learning

ACM Reference Format:

Mengduo Yang, Yi Yuan, Jie Zhou, Meng Xi, Xiaohua Pan, Ying Li, Yangyang Wu, Jinshan Zhang, and Jianwei Yin. 2024. Adaptive Fusion of Multi-View for Graph Contrastive Recommendation. In *18th ACM Conference on Recommender Systems (RecSys '24)*, October 14–18, 2024, Bari, Italy. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3640457.3688153>

1 INTRODUCTION

Recently, recommender systems have emerged as essential services that offer personalized item recommendations to users, aiming to mitigate the issue of information overload on the internet. To address this issue, collaborative filtering (CF) assumes that users with similar behaviors tend to have similar preferences for items, which remains a fundamental task in achieving effective personalized recommendations [1, 26, 29].

*Both authors contributed equally to this research.

†Meng Xi is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '24, October 14–18, 2024, Bari, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0505-2/24/10

<https://doi.org/10.1145/3640457.3688153>

To implement the assumption of CF, a commonly utilized strategy is to reconstruct historical interactions by assigning embeddings to both users and items, which in turn facilitates the prediction of user preferences with these embeddings. Matrix factorization (MF) is an early such model, which directly projects the single ID of a user to his/her embedding [9]. Then, inspired by the power of deep learning, several research studies extend MF embedding function by introducing neural networks for improving the quality of embedding [6, 7]. Later on, to model the high-order connectivity information in the embedding function, graph neural networks (GNNs) are utilized in collaborative filtering for enhancing the user/item representation [2, 5, 21, 23]. Recently, some studies show that graph contrastive learning (GCL) methods have achieved great success in recommendation, which integrate graph neural networks with contrastive learning by propagating and aggregating augmented user/item representation in contrastive views [1, 13, 26, 33].

Although state-of-the-art GCL-based methods have achieved remarkable success in recommendation, we argue that they suffer from several inherent limitations. First, most GCL-based methods leverage graph augmentation strategies (e.g., random structure/embedding perturbation) to generate contrastive views, and cannot dynamically update the contrastive views during the learning process. This practice may bring noise to representation learning and is detrimental when the generated views are erroneous. Second, these methods utilize a static Laplacian operator for GNN-based representation propagation and aggregation, thus disregarding the users' preference degree on items, which results in an incomplete user/item modeling.

To address the limitations mentioned above, we propose the Adaptive Fusion of Multi-View Graph Contrastive Recommendation (AMGCR) model by incorporating graph contrastive learning with learnable view generation and fusion. Specifically, we design four learnable generators to obtain the informative edge weights for constructing contrastive views, focusing on weight adjustment, feature transformation, neighbor aggregation, and attention mechanisms, respectively. Then, we employ an adaptive multi-view fusion module to effectively combine the views obtained by the four view generators. In addition, to make the model capable of capturing preference information during the learning process, we further adopt a preference refinement strategy on the combined contrastive view, thus empowering the model's ability to capture the preferences between users and items during the contrastive learning process. Finally, we employ a multi-task learning strategy to assist in the learning of the recommendation task to achieve better performance. To summarize, our contributions are mainly as follows:

- We design four view generators to learn the edge weights in the interaction graph for contrastive view instead of random perturbation, which significantly alleviates the problem of noise introduction. Moreover, we utilize an adaptive fusion strategy to combine these edge weights by considering both view-shared and view-specific levels.
- We propose a preference refinement mechanism to make the model capable of capturing preference between users and items during the contrastive learning process, which further enhances the learned representation.

- Extensive experiments conducted on three real-world benchmark datasets demonstrate that our method surpasses more than 10 existing methods. Experimental analysis further validates model effectiveness, indicating its potential as a reference for practitioners in the field.

2 RELATED WORK

2.1 GNN-based Recommendation

Graph neural networks (GNNs) have shown powerful performance in learning representation over graph structures for resolving downstream tasks. Motivated by the achievements of GNNs, researchers have developed novel graph neural architectures tailored for CF-based recommendation [4, 27]. These architectures aim to capture the high-order connectivity information on user-item interaction graph by employing powerful GNN models. For example, GCMC [16] proposes a graph auto-encoder framework based on differentiable message passing on the bipartite interaction graph to perform matrix completion for recommender systems. NGCF [21] injects collaborative signals into interaction modeling based on GNN, to capture higher-order connectivity patterns. STGCN [34] designs a new STacked and Reconstructed Graph Convolutional Networks architecture to learn node representations for boosting the performance in recommender systems, especially in the cold start scenario. Later on, considering scalability enhancement and mitigation of over-smoothing issue, approaches like GCCF [2], LightGCN [5] and HGCF [15] have optimized the architecture of GNNs in collaborative filtering. For example, LightGCN simplifies GNN for recommendation by removing the non-linear activation as well as the feature transformation layer, and now it is a popular collaborative filtering method. In addition, SimRec [28] is proposed to adaptively transfer knowledge from the GNN-based teacher model to a small feed-forward network.

However, in recommendation scenarios, the problem of data sparsity poses a challenge that traditional GNN-based methods often struggle to overcome.

2.2 Contrastive Learning in Recommendation

Recently, with the ability to address the issue of data noise and label sparsity, contrastive learning (CL) [20] has gained considerable attention in various recommendation scenarios, such as sequential recommendation [3], knowledge graph-based recommendation [36], multi-interest recommendation [35] and multi-behavior recommendation [25]. In the realm of collaborative filtering recommendations, one CL-based prevalent practice is to incorporate graph neural networks with contrastive learning, by learning user/item representation in contrastive views. For example, SGL [26] performs data augmentation over graph structure with random dropout operations to generate different views. However, such stochastic augmentation may drop important information, which may make the sparsity issue of inactive users even worse. Hence, some recent graph contrastive learning (GCL) methods are proposed to address this issue [19]. Particularly, HCCF [29] and NCL [11], design heuristic-based strategies to construct views for embedding contrasting. SimGCL [33] experimentally determines that the contribution of data augmentation to SGL is minimal, thus

proposing the addition of noise to each layer embedding to generate positive views. Based on SimGCL, XSimGCL [32] discards the ineffective augmentations and instead employs a noise-based embedding augmentation for contrastive Learning, with greatly reducing the complexity of GCL-based approaches.

Despite their impressive performance, they are still susceptible to introducing noise due to random perturbation on graph structure or embeddings, making it difficult to adapt to recommendation tasks in different scenarios.

3 PRELIMINARIES

3.1 Problem Formulation

In recommendation scenario, we use $\mathcal{U} = \{u_1, u_2, \dots, u_M\}$ and $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ to denote the sets of users and items, respectively, where M, N are the number of users and items. The user-item interaction matrix $\mathbf{A} \in \mathbb{R}^{M \times N}$ represents the observed interactions between users and items, where an element $A_{i,j} = 1$ if user u_i has interacted with item v_j , otherwise $A_{i,j} = 0$. We use an array $\mathcal{E}$ of size $(2, E)$ to denote the linked users and items in interaction graph, where E is the number of edges. For example, user $\mathcal{E}_{0,c}$ and item $\mathcal{E}_{1,c}$ are linked in the interaction graph, where $c \in \{0, 1, \dots, E-1\}$. Then, the goal of recommendation can be formalized as:

$$f_{\theta}(\mathbf{A}_{i,:}) \implies \mathcal{R}_i, \quad (1)$$

where f is the mapping function with parameters θ , and $\mathcal{R}_i$ denotes the items that user i tends to access next.

3.2 GNN-based Interaction Graph Learning

In neural collaborative filtering, both users and items are associated with learnable embedding vectors $\mathbf{E}^u \in \mathbb{R}^{M \times d}$ and $\mathbf{E}^v \in \mathbb{R}^{N \times d}$, where d is the embedding dimension. Given the user i and item j , their embedding are denoted as e_i^u and e_j^v respectively. In order to learn the valuable contextual information from the original relationships, GNN-based methods often employ L GNN layers to propagate and aggregate user (item) representation on the interaction graph. Specifically, the l -th GNN layer is defined as:

$$z_{i,l}^u = \sigma(\tilde{\mathbf{A}}_{i,:} \mathbf{Z}_{l-1}^v), \quad z_{j,l}^v = \sigma((\tilde{\mathbf{A}}_{:,j})^T \mathbf{Z}_{l-1}^u), \quad (2)$$

where $z_{i,l}^u$ and $z_{j,l}^v$ represent the l -th layer output representation for user i and item j . We initialize $\mathbf{Z}_0^u$ and $\mathbf{Z}_0^v$ with $\mathbf{E}^u$ and $\mathbf{E}^v$, respectively. $\sigma(\cdot)$ denotes the LeakyReLU activation function. $\tilde{\mathbf{A}}$ is the normalized interaction matrix, where $\tilde{A}_{i,j} = A_{i,j} / (D_i^u D_j^v)^{1/2}$, $D_i^u = \sum A_{i,:}$ and $D_j^v = \sum A_{:,j}$. Then we sum the output representation from all the GNN layers to obtain the final user (item) representation as follows:

$$z_i^u = \sum_{l=0}^L z_{i,l}^u, \quad z_j^v = \sum_{l=0}^L z_{j,l}^v, \quad (3)$$

where $z_i^u \in \mathbf{Z}^u$ and $z_j^v \in \mathbf{Z}^v$ are the learned representation of user i and item j from the user-item interaction graph, respectively.

4 THE PROPOSED FRAMEWORK

The architecture of AMGCR is shown in Figure 1. First, we design four view generators to learn edge weights for contrastive view

construction. Then, we employ the adaptive multi-view fusion module to combine the edge weights to get the contrastive view, and further refine it with the preference matrix to enable the model to capture preference information during the contrastive learning process, facilitating better representation learning. Finally, we perform contrastive learning to optimize the learned representation, which assists in the learning of the main recommendation task to achieve better performance.

4.1 Learnable View Generators

Existing graph contrastive learning (GCL)-based methods often generate contrastive views by perturbing graph structure or node embeddings. This may introduce a significant amount of noise or discard important information, thus leading to lower efficiency and suboptimal performance for contrastive learning. Therefore, we propose a learnable approach to capture useful information on the graph and obtain informative learnable views for efficient and effective contrastive learning. It is worth noting that instead of perturbing edges or embeddings, we aim to learn the weight of existing edges $\mathcal{W} \in \mathbb{R}^E$ in the interaction graph and utilize these weights to construct contrastive views. In this way, the contrastive views are able to capture important relationship information, which ensures the generated view is informative and less noisy. Specifically, we design four learnable view generators as follows.

4.1.1 Weight-vector-based View Generator. To facilitate the adjustment of weights during training and effectively capture crucial patterns and dependencies, we introduce a learnable weight vector $\mathbf{W} \in \mathbb{R}^E$ to denote the learnable weights associated with the edges, where E represents the number of edges in the original interaction graph $\mathbf{A}$. We initialize $\mathbf{W}$ as a vector of all ones, yielding the weight-vector-based edge weights denoted as $\mathcal{W}^{WV} = \mathbf{W}$. This approach allows for dynamic adaptation of edge weights, enabling the model to better discern and incorporate significant features and relationships within the graph structure.

4.1.2 MLP-based View Generator. To further capture higher-order embedding interactions and enable flexible combinations of embeddings, we augment the expressive capacity of the generator using multilayer perceptron (MLP) architectures. This enhancement leads to the derivation of $\mathcal{W}^{MLP}$, which facilitates the exploration and integration of intricate relationships among embeddings. By leveraging MLPs, the model gains the capability to learn complex patterns and adaptively adjust embedding interactions, thereby enhancing the model's representational power and effectiveness in capturing nuanced dependencies within the data. Specifically, we obtain $\mathcal{W}^{MLP}$ as:

$$\mathcal{W}_c^{MLP} = \phi \left(\delta \left(\text{MLP}(e_{\mathcal{E}_{0,c}}^u) \right), \delta \left(\text{MLP}(e_{\mathcal{E}_{1,c}}^v) \right) \right), \quad (4)$$

where $\phi(\cdot, \cdot)$ denotes similarity function (i.e., inner product) and $\delta(\cdot)$ is nonlinear activation function.

4.1.3 GNN-based View Generator. The original structural information within the interaction graph holds significant value, serving as a guiding force for generating less noisy views. Therefore, we leverage the learned representations derived from the original user-item interaction graph ($z_i^u \in \mathbf{Z}^u$ and $z_j^v \in \mathbf{Z}^v$ as outlined in section

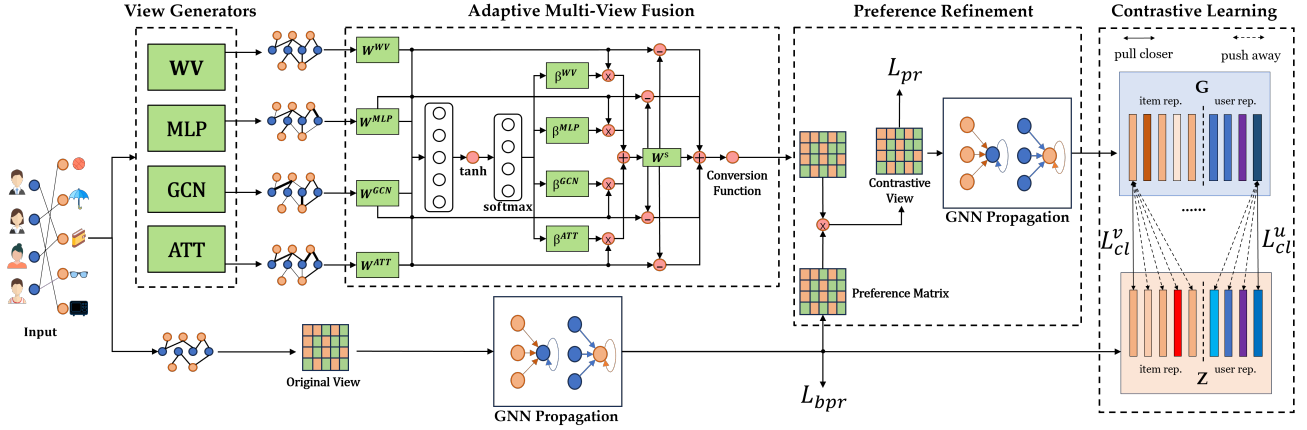


Figure 1: The overview of the proposed model AMGCR.

3.2) to construct GNN-based edge weights $\mathcal{W}^{GCN}$, formulated as:

$$\mathcal{W}_c^{GCN} = \phi(z_{\mathcal{E}_{0,c}}^u, z_{\mathcal{E}_{1,c}}^v), \quad (5)$$

which enables the incorporation of learned structural features into the edge weights, facilitating the generation of more informative and contextually relevant views.

4.1.4 Attention-based View Generator. Motivated by the principles outlined in Graph Attention Networks (GAT) [18], where the relevance of different items to users may vary significantly, we integrate an attention mechanism to discern and prioritize crucial interactions, thereby enhancing the quality of generated views. Specifically, we denote $i = \mathcal{E}_{0,c}$, $j = \mathcal{E}_{1,c}$, and compute $\mathcal{W}^{ATT}$ as:

$$\alpha_{i,j} = \delta((g^T e_i^u) \cdot (g^T e_j^v)),$$

$$\mathcal{W}_c^{ATT} = \frac{e^{(\alpha_{i,j})}}{\sum_{A_{i,k}=1} e^{(\alpha_{i,k})}}, \quad (6)$$

where $\alpha_{i,j}$ is the normalized attention coefficient between user i and item j . $g \in \mathbb{R}^d$ is a learnable weight vector.

4.2 Adaptive Multi-View Fusion

By utilizing the above designed view generators, we could obtain four kinds of edge weights for contrastive view construction. They are calculated to assess the relationship between users and items from four different perspectives, i.e., weight adjustment, feature transformation, neighbor aggregation, and attention mechanism, which have all been proven effective in subsequent experiments. Therefore, to effectively fuse these edge weights in one perfect contrastive view, we design an Adaptive Multi-View Fusion module, which allows flexible fusion based on different kinds of edge weights. The core idea of this module is to decouple the view-shared edge weights and integrate them with the view-specific edge weights.

4.2.1 View-shared Edge Weights. The view-shared edge weights are crucial, as these edge weights are acknowledged across different views, which demonstrates that such edge weights effectively reflect the relationship between users and items. For this reason, we first extract the view-shared edge weights through attention

mechanisms, where the attention weight of the different kinds of view $K = \{WV, MLP, GCN, ATT\}$ ($k \in K$) are calculated as:

$$\beta_k = \text{softmax}(\tanh(\mathbf{W}_1 \odot \mathcal{W}^k + \mathbf{b}_1)), \quad (7)$$

where $\mathbf{W}_1 \in \mathbb{R}^E$ and $\mathbf{b}_1 \in \mathbb{R}^E$ denote the weight vector and the bias vector, respectively. Notice that these parameters are shared for the all kinds of edge weights, thus the view-shared edge weights are obtained as:

$$\mathcal{W}^s = \sum_{k \in K} \beta_k \mathcal{W}^k, \quad (8)$$

where $\mathcal{W}^s$ is the view-shared edge weights across the four views.

4.2.2 View-specific Edge Weights. For the view-specific edge weights, user attention for items remains unique, as this aligns with the different purpose of the user's intent. Here, we obtain the view-specific edge weights by subtracting the view-shared edge weights as follows:

$$\bar{\mathcal{W}}^k = \mathcal{W}^k - \mathcal{W}^s, \quad (9)$$

where $\bar{\mathcal{W}}^k$ is the view-specific edge weights for k -based view.

4.2.3 Fusion of the Edge Weights. Then, we fuse the view-specific edge weights, and combine them with view-shared edge weights to obtain the final edge weights as:

$$\mathcal{W}^f = \mathcal{W}^s + \sum_{k \in K} \bar{\mathcal{W}}^k. \quad (10)$$

Finally, we generate the contrastive graph based on the given edge indices $\mathcal{E}$ and final edge weights as:

$$\mathbf{A}^f = \Phi(\mathcal{W}^f, \mathcal{E}), \quad (11)$$

where $\mathbf{A}^f$ is the obtained contrastive view. $\Phi(\cdot, \cdot)$ creates a sparse FloatTensor of the graph based on the edge indices, the learned edge weights, and the dimensions of the tensor. The specific code implementation can be found in our code repository.

4.3 Preference Refinement

Current methods often utilize a static normalized adjacency matrix to perform GNN-based representation learning, which disregards the user's preference on items, resulting in an incomplete user/item

Table 1: Comparisons of computational complexity with baselines. In the table, E , L and d denote the number of edges, the number of GNN layers, and the embedding size; $\rho \in (0, 1]$ is the edge keep rate in SGL; q , K_1 and K_2 are the SVD-required rank in LightGCL, the number of latent intents in DCCF and the number of prototypes in NCL. M and N represent the number of users and items; B and T are the batch size and node number in a batch.

Stage	LightGCN	SGL	LightGCL	DCCF	NCL	SimGCL	XSimGCL	AMGCR
Pre-processing	$O(E)$	$O(E)$	$O((q+1)E)$	$O(E)$	$O(E)$	$O(E)$	$O(E)$	$O(E)$
Augmentation	-	$O(2\rho E)$	-	$O(2ELd)$	$O((M+N)d)$	$O((M+N)d)$	$O((M+N)d)$	$O(4Ed)$
GNN	$O(2ELd)$	$O(2ELd + 4\rho ELd)$	$O(2ELd + 2q(M+N)Ld)$	$O(2ELd + K_1(M+N)Ld)$	$O(2ELd)$	$O(6ELd)$	$O(2ELd)$	$O(4ELd)$
BPR Loss	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$	$O(2Bd)$
Contrastive Loss	-	$O(Bd + BTd)$	$O((Bd + BTd)L)$	$O(B(M+N)Ld)$	$O(Bd + B(K_2 + T)d)$	$O(Bd + BTd)$	$O(Bd + BTd)$	$O(Bd + BTd)$

modeling. Hence, to make the model capable of capturing preference information during the learning process, we adopt a preference refinement strategy to make the contrastive view maintain the preference information. Specifically, we compute the preference matrix $\mathbf{P}$, where $p_{i,j}$ denotes the preference of user i for item j , as follows:

$$p_{i,j} = \begin{cases} \delta(z_i^u \cdot z_j^v) & , \text{ if } \mathbf{A}_{i,j} = 1 \\ 0 & , \text{ otherwise} \end{cases} \quad (12)$$

where z_i^u and z_j^v are learned representation from the original view in Eq. 3. The preference matrix $\mathbf{P}$ measures the preference of users for items by comprehensively considering both graph structure and node representation. The preference degree of users for items and the interaction probability could be reflected by $\mathbf{P}$. Therefore, we utilize $\mathbf{P}$ to refine the contrastive view $\mathbf{A}^f$ with Hadamard product:

$$\tilde{\mathbf{A}}^f = \mathbf{A}^f \odot \mathbf{P}, \quad (13)$$

where $\tilde{\mathbf{A}}^f$ is the refined contrastive view. In order to further refine the contrastive view and preserve the original interaction information, we introduce a preference refinement loss as:

$$\mathcal{L}_{pr} = \sum_{\mathbf{A}_{i,j}=1} -\log \tilde{\mathbf{A}}_{i,j}^f. \quad (14)$$

The purpose of this loss is to encourage the interaction probabilities between interacted user-items to be high, thus preserving the original structural information and ensuring that the contrastive view is well constrained. By this setting, the reliability of the generated learnable view $\tilde{\mathbf{A}}^f$ is improved, and the preference information can be well modeled and captured. Then we define the graph propagation and aggregation on the refined contrastive view as:

$$\begin{aligned} \mathbf{G}_l^u &= \sigma(\tilde{\mathbf{A}}^f \mathbf{G}_{l-1}^v), \mathbf{G}_l^v = \sigma((\tilde{\mathbf{A}}^f)^T \mathbf{G}_{l-1}^u), \\ \mathbf{G}^u &= \sum_{l=0}^L \mathbf{G}_l^u, \mathbf{G}^v = \sum_{l=0}^L \mathbf{G}_l^v, \end{aligned} \quad (15)$$

where we initialize $\mathbf{G}_0^u$ and $\mathbf{G}_0^v$ with the embedding $\mathbf{E}^u$ and $\mathbf{E}^v$, respectively. $\mathbf{G}^u$ and $\mathbf{G}^v$ are the learned representation of users and items from the contrastive view.

4.4 Contrastive Learning

Most existing GCL methods contrast node representation from two extra views, while the representation learned from the main view (the original view) is not directly involved in InfoNCE loss [17]. This contrastive learning strategy is adopted due to the concern that the random perturbation view could potentially introduce

misleading signals to the main view representation. In contrast, we construct the contrastive view based on the learned edge weights and adaptive fusion, and we incorporate the preference information, which is informative and less noisy. Hence, we design a contrastive learning task by contrasting the generated view representation g_i^u (from $\mathbf{G}^u$) with the original view representation z_i^u (from $\mathbf{Z}^u$) as:

$$\mathcal{L}_{cl}^u = \sum_{i=1}^M -\log \frac{\exp(\phi(z_i^u, g_i^u)/\tau)}{\sum_{i'=1}^M \exp(\phi(z_i^u, g_{i'}^u)/\tau)} \quad (16)$$

where τ denotes the temperature, which is used to adjust the probability distribution of the output. The InfoNCE loss $\mathcal{L}_{cl}^v$ for the items are defined in the same way.

4.5 Inference and Optimization

4.5.1 Inference. We adopt an inner product to predict the interaction probability $\hat{y}_{i,j}$ between user i and item j as:

$$\hat{y}_{i,j} = z_i^u \cdot z_j^v. \quad (17)$$

4.5.2 Optimization. We employ a multi-task training approach to train our proposed model, which involves optimizing the main BPR recommendation task, the contrastive learning task, and the preference refinement task. Specifically, we define the overall objective loss function $\mathcal{L}$ as follows ($\hat{y}_{i,j}$ and $\hat{y}_{i,n}$ denote the predicted scores for a pair of positive and negative items of user i):

$$\begin{aligned} \mathcal{L}_{cl} &= \mathcal{L}_{cl}^u + \mathcal{L}_{cl}^v, \mathcal{L}_{bpr} = -\sum_{i=1}^M \log \delta(\hat{y}_{i,j} - \hat{y}_{i,n}), \\ \mathcal{L} &= \mathcal{L}_{bpr} + \lambda_1 \mathcal{L}_{cl} + \lambda_2 \mathcal{L}_{pr} + \lambda_3 \|\Theta\|_2^2, \end{aligned} \quad (18)$$

where λ_1 , λ_2 and λ_3 are the weights for contrastive loss, preference refinement loss and regularization loss, respectively.

4.6 Time Complexity Analysis

In this section, we analyze the time complexity of our model during both the pre-processing and the batch training stages. We also compare it with the state-of-art methods as presented in Table 1. We assume that for each positive interaction, only one negative interaction is sampled.

4.6.1 Complexity of the Pre-processing. All the methods need to normalize the adjacency matrix, which has an $O(E)$ time complexity. Note that LightGCL needs to perform Singular Value Decomposition (SVD) to generate its contrastive view, which leads to an extra computational cost of $O(qE)$.

4.6.2 Complexity of the Batch Training.

- **Augmentation.** Our method generates four kinds of edge weights based on the interaction graph, fuses them adaptively and refines the view based on preferences, which leads to an $O(4Ed)$ time complexity. For the other methods, SGL performs two graph augmentations, which is $O(2pE)$; DCCF requires distill latent intent factors across all layers, which is $O(2ELd)$. NCL need perform clustering to get the prototypes, which is $O((M+N)d)$; Both SimGCL and XSimGCL add noise on the learned embeddings, which are also $O((M+N)d)$.
- **GNN.** Our method performs GNN on both original view and contrastive view, which is an $O(4ELd)$ time complexity. A three-GNN architecture is adopted in both SGL and SimGCL to learn two different augmentations, which is $O(6ELd)$.
- **BPR Loss.** All methods are trained with the BPR loss and each batch contains B interactions, with B random negative interaction, so they have exactly the same time cost in this regard.
- **Contrastive Loss.** Our method contrasts the positive with negative samples in a batch, which is $O(Bd + BTd)$ complexity; DCCF contrasts all nodes across all layers per-batch, thus leading to $O(B(M+N)Ld)$ complexity; NCL need perform K_2 -prototype contrastive learning, which brings extra $O(BK_2d)$ complexity.

To sum up, XSimGCL is the current lightweight yet effective method. And compared to all these methods, our AMGCR demonstrates significant competitiveness in terms of time complexity.

5 EVALUATION

In order to verify the effectiveness of AMGCR, we try to answer the following research questions:

- **RQ1:** Compared with state-of-the-art methods, how effective is AMGCR to boost recommendation?
- **RQ2:** In comparison to state-of-the-art methods, how efficient is the performance of AMGCR?
- **RQ3:** Do the proposed primary components enhance recommendation performance of AMGCR?
- **RQ4:** How does the model capture the varying degrees of user preference for different items?
- **RQ5:** How do the different hyper-parameter settings affect the model performance?

Table 2: Description of datasets used in evaluations.

Dataset	#Users	#Items	#Interactions	Density
Gowalla	50,821	57,440	1,172,425	4.0e-4
Amazon-book	78,578	77,801	2,240,156	3.7e-4
Tmall	47,939	41,390	2,357,450	1.2e-3

5.1 Experimental Setup

5.1.1 Datasets. We evaluate our methods on three public datasets derived from real-world applications [1], including Gowalla, Amazon-book, and Tmall. **Gowalla** dataset is collected from the Gowalla platform, which captures the check-in relationships between users and diverse locations. **Amazon-book** dataset encompasses users' rating behaviors for products falling under the book category on

the Amazon platform. **Tmall** dataset focuses on customer purchase behaviors observed on the online retailer Tmall. Following the data preprocessing process in the method DCCF [13], we extracted user-item interactions from the raw dataset and then divided the extracted dataset into training, validation, and testing sets with a ratio of 7:2:1. Detailed statistics of the datasets are presented in Table 2.

5.1.2 Baselines. To evaluate the proposed recommendation framework of AMGCR, we compare with the state-of-the-art recommendation baselines presented in three groups.

(i) General neural methods.

- NCF [7]. It utilizes a multi-layer perceptron instead of inner product in matrix factorization to estimate user-item interactions.
- AutoR [14]. It employs an autoencoder to reconstruct user-item interactions and obtains user interests for non-interacted items.

(ii) GNN-based methods.

- NGCF [21]. It injects collaborative signals into interaction modeling based on GNN, to capture higher-order connectivity patterns.
- LightGCN [5]. It is a popular collaborative filtering method based on NGCF, which simplifies GNN for recommendation.
- DisenGCN [12]. It learns decoupled node representations via a neighborhood routing mechanism and predicts interaction likelihood between users and items using dot products.
- DisenHAN [24]. It disentangles user/item representations into distinct aspects or latent intents and aggregates information from these aspects using attention mechanisms.
- DGCF [22]. It models a distribution over intents, which learns disentangled representations from the intent-aware graph.

(iii) CL-based methods.

- SLRec [31]. It introduces a multi-task self-supervised learning framework, addressing label sparsity in large-scale RS.
- SGL [26]. It augments user/item representation learning with GNN and incorporates self-supervised contrastive learning.
- DGCL [10]. It employs a factor-wise discrimination objective and learns disentangled representations of nodes and predicts user-item interactions using inner products.
- HCCF [29]. It jointly captures local and global collaborative signals using a hyper-graph neural network and integrates cross-view contrastive learning for data augmentation.
- NCL [11]. It considers similar semantic prototypes and structural neighbors as positive views for contrastive learning.
- LightGCL [1]. It is a lightweight graph CL framework that leverages singular value decomposition to generate augmented views.
- DCCF [13]. It is a decoupled CL-based recommendation method that models latent factors underlying interactions.
- SimGCL [33]. It proposes the addition of noise to each layer embedding to generate positive instances for contrastive learning.
- XSimGCL [32]. It discards the ineffective augmentations and instead employs a noise-based embedding augmentation for CL.

5.1.3 Implementations and Metrics. We implement all the methods in Pytorch and Adam Optimizer with learning rate of 0.001. For a fair comparison, we refer to the best hyper-parameter settings reported in the original papers of the baselines and then fine-tune them with the grid search. As for the general settings, we set the

Table 3: Comparing various state-of-the-art recommendation baselines on benchmark datasets.

Datasets	Gowalla				Amazon-book				Tmall			
Metrics	Recall@20	Recall@40	NDCG@20	NDCG@40	Recall@20	Recall@40	NDCG@20	NDCG@40	Recall@20	Recall@40	NDCG@20	NDCG@40
NCF	0.1247	0.1910	0.0659	0.0832	0.0468	0.0771	0.0336	0.0438	0.0383	0.0647	0.0252	0.0344
AutoR	0.1409	0.2142	0.0716	0.0905	0.0546	0.0914	0.0354	0.0482	0.0336	0.0611	0.0203	0.0295
NGCF	0.1413	0.2072	0.0813	0.0987	0.0532	0.0866	0.0388	0.0501	0.0420	0.0751	0.0250	0.0365
LightGCN	0.1799	0.2577	0.1053	0.1255	0.0732	0.1148	0.0544	0.0681	0.0555	0.0895	0.0381	0.0499
DisenGCN	0.1379	0.2003	0.0798	0.0961	0.0481	0.0776	0.0353	0.0451	0.0422	0.0688	0.0285	0.0377
DisenHAN	0.1437	0.2079	0.0829	0.0997	0.0542	0.0865	0.0407	0.0513	0.0416	0.0682	0.0283	0.0376
DGCF	0.1784	0.2515	0.1069	0.1259	0.0688	0.1073	0.0513	0.0640	0.0544	0.0867	0.0372	0.0484
SLRec	0.1529	0.2200	0.0926	0.1102	0.0544	0.0879	0.0374	0.0490	0.0549	0.0888	0.0375	0.0492
SGL	0.1833	0.2627	0.1090	0.1294	0.0844	0.1317	0.0653	0.0799	0.0641	0.0982	0.0440	0.0561
DGCL	0.1793	0.2483	0.1067	0.1247	0.0677	0.1057	0.0506	0.0631	0.0526	0.0845	0.0359	0.0469
HCCF	0.1818	0.2601	0.1061	0.1265	0.0824	0.1282	0.0625	0.0776	0.0623	0.0986	0.0425	0.0552
NCL	0.1831	0.2624	0.1089	0.1293	0.0846	0.1318	0.0656	0.0802	0.0647	0.0986	0.0446	0.0568
LightGCL	0.1825	0.2601	0.1077	0.1280	0.0836	0.1280	0.0643	0.0790	0.0632	0.0971	0.0444	0.0562
DCCF	0.1876	0.2644	0.1123	0.1323	0.0889	0.1343	0.0680	0.0829	0.0668	0.1042	0.0469	0.0598
SimGCL	0.1927	0.2699	0.1139	0.1344	0.0907	0.1365	0.0696	0.0839	0.0680	0.1053	0.0480	0.0609
XSimGCL	0.1933	0.2709	0.1145	0.1350	0.0911	0.1368	0.0701	0.0844	0.0693	0.1072	0.0489	0.0621
AMGCR	0.2170	0.3018	0.1320	0.1541	0.1063	0.1577	0.0828	0.0996	0.0772	0.1184	0.0546	0.0689
Imprv.	↑ 12.3%	↑ 11.4%	↑ 15.3%	↑ 14.1%	↑ 16.7%	↑ 15.2%	↑ 18.1%	↑ 18.0%	↑ 11.4%	↑ 10.4%	↑ 11.7%	↑ 10.9%
p-val.	$3.6e^{-4}$	$1.4e^{-4}$	$2.3e^{-5}$	$7.4e^{-5}$	$5.6e^{-6}$	$1.8e^{-4}$	$9.2e^{-7}$	$4.5e^{-6}$	$3.7e^{-5}$	$8.1e^{-5}$	$6.0e^{-7}$	$0.2e^{-7}$

initial embedding size $d = 32$, the batchsize $B = 4096$, and set the temperature $\tau = 0.2$, which is often reported a great choice in papers on CL [26]. For GNN-based methods, the number of GNN layers L is chosen from $\{1, 2, 3\}$. When reproducing certain baseline methods, we use the hyper-parameter values as published in the original papers, with some adjustments to attain optimal results. For our method, λ_1 , λ_2 and λ_3 are tuned from the range of $[0.1, 0.5]$, $[0, 1.0]$, and $[1e-7, 1e-5]$ respectively. We perform an all-rank evaluation, where positive items from the test set are ranked alongside all un-interacted items for each user [21, 30], and adopt two widely used top-K metrics, Recall@K and NDCG@K [8, 26] with $K = \{20, 40\}$. All experiments are conducted on a machine with Linux OS, featuring a 32-core CPU, a 24GB NVIDIA 4090 GPU, and 125GB memory.

5.2 Performance Analysis (RQ1)

To answer research question **RQ1**, we list the performance comparison of different recommendation methods in Table 3. We make the following major observations.

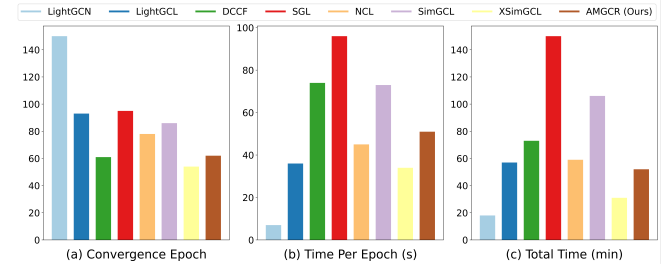
① *AMGCR consistently outperforms all baselines on all datasets.* AMGCR gains the great improvements against the baselines. Although GCL-based methods have achieved promising performance, our AMGCR still outperforms them significantly. One key reason is that our model utilizes the learnable augmentation on the edge weights for contrastive view generation, which is able to capture richer semantic information and change dynamically to keep the view effective. Through the adaptive fusion of the learned edge weights, the obtained contrastive view is more effective than the single. Moreover, the preference refinement strategy enable the model to capture user preference for different items during the learning process, which further enhances the learned representation.

② *AMGCR achieves a greater improvement in Recall@20 (NDCG@20) compared to Recall@40 (NDCG@40).* Our experiments have shown that our model yields a more notable improvement in Metric@20 than in Metric@40. This suggests that our model is better at ranking

items of interest for users within the top 20 recommended results, while the top 40 recommendations may include more items that users are not interested in. These findings also suggest that AMGCR is more effective in capturing user interests. Overall, our results indicate the effectiveness of our model in accurately ranking user-relevant items within the top 20 recommendations, demonstrating its ability to understand and predict user interests while effectively filtering out unrelated items.

5.3 Efficiency Study (RQ2)

In this section, we analyze the batch training efficiency of our model in comparison to seven competitive baselines, as shown in Figure 2. Specifically, we report the actual training time on the Gowalla dataset and make the following observations.

**Figure 2: The training time of different methods.**

① *For the convergence epoch and per-batch training time, AMGCR is comparable to the state-of-the-art GCL-based methods.* No-GCL method LightGCN is trained with hundreds of epochs. By contrast, GCL-based methods (SGL, LightGCL, DCCF, NCL, SimGCL and XSimGCL) need the fewest epochs to reach convergence, due to the contrastive learning for accelerating convergence. AMGCR ranked third in the number of epochs to get converged. Consistent

with our analysis on time complexity in section 4.6, our AMGCR approach ranks third in batch training time among GCL-based methods, following behind NCL and XSimGCL.

② *The total training time of AMGCR is also competitive compared with the existing Sota methods.* SGL takes the longest time to finish the computation in a single batch, which is almost several times that of LightGCN on all the datasets. XSimGCL ranks second due to its lightweight contrastive learning architecture. Our AMGCR ranks third due to its effective and efficient contrastive view for better contrastive learning, and shows a significant improvement in recommendation performance.

Table 4: Performance with different variants (@20).

Variants	Metric	Gowalla	Amazon-book	Tmall
Only Main	Recall	0.1799±0.002	0.0732±0.001	0.0555±0.001
	NDCG	0.1053±0.003	0.0544±0.002	0.0381±0.001
w/ WV	Recall	0.2112±0.003	0.1036±0.004	0.0728±0.001
	NDCG	0.1276±0.001	0.0808±0.002	0.0517±0.002
w/ MLP	Recall	0.2138±0.002	0.1032±0.002	0.0734±0.001
	NDCG	0.1296±0.001	0.0805±0.002	0.0522±0.001
w/ GNN	Recall	0.2124±0.003	0.1041±0.001	0.0744±0.001
	NDCG	0.1284±0.002	0.0812±0.002	0.0528±0.002
w/ ATT	Recall	0.2132±0.004	0.1043±0.003	0.0736±0.001
	NDCG	0.1293±0.002	0.0814±0.002	0.0523±0.001
w/ Sum Fusion	Recall	0.2141±0.001	0.1039±0.002	0.0748±0.001
	NDCG	0.1298±0.004	0.0810±0.002	0.0530±0.001
w/ Shared	Recall	0.2148±0.002	0.1045±0.002	0.0754±0.002
	NDCG	0.1301±0.003	0.0813±0.001	0.0534±0.001
w/ Specific	Recall	0.2152±0.003	0.1042±0.002	0.0750±0.001
	NDCG	0.1305±0.002	0.0811±0.003	0.0531±0.001
w/ Ada Fusion (AMGCR)	Recall	0.2170±0.003	0.1063±0.002	0.0772±0.001
	NDCG	0.1320±0.002	0.0828±0.003	0.0546±0.001
w/o PR	Recall	0.2106±0.003	0.1013±0.002	0.0725±0.001
	NDCG	0.1272±0.003	0.0794±0.002	0.0515±0.001

5.4 Ablation Study (RQ3)

In this section, we conduct extensive experiments to verify the effectiveness of the components in AMGCR. We present the four view generators as WV, MLP, GNN and ATT, respectively. Sum Fusion denotes the simple fusion of the four views based on sum operation, w/ Shared denotes only using the shared weights, w/ Specific denotes only using the specific weights, while Ada Fusion denotes the proposed adaptive multi-view fusion. PR represents the proposed preference refinement. only Main denotes the BPR model with GNN, which can be regraded as the model LightGCN. Table 4 reveals the performance of the different variants of AMGCR, and we make the following observations.

① *The performance of different kinds of edge weights varies across different datasets.* For instance, on the Gowalla dataset, w/ MLP exhibits the best performance, while w/ ATT performs the best on the Amazon-book dataset, and w/ GNN demonstrates superior performance on the Tmall dataset. This indicates that the nature of different datasets varies, and hence, the suitable kinds of edge

weights also differ. Therefore, to obtain edge weights adaptable to all datasets, we propose an adaptive multi-view view fusion module.

② *The adaptive multi-view fusion module make the fused view more suitable on all datasets.* We observe that the simple fusion of different kinds of edge weights, i.e., summation, often outperforms four individual view. We also find that directly using the fusion view via shared and specific weights resulted in decreased performance. This can be explained by the fact that each view contains shared and unique information that needs to be comprehensively considered to achieve better results. Our proposed adaptive fusion approach yields the best results, demonstrating its superiority. This underscores the effectiveness of our fusion method, which fully considers both view-shared and view-specific edge weights across views, resulting in more effective fusion process.

③ *Preference refinement strategy further enhances performance.* By removing preference refinement, we create the variant w/o PR. We find that w/o PR achieves inferior performance compared to the AMGCR, which highlights the benefits of preference refinement in making the contrastive view to maintain useful preference information for better contrastive learning, and providing richer semantic information during the training process.

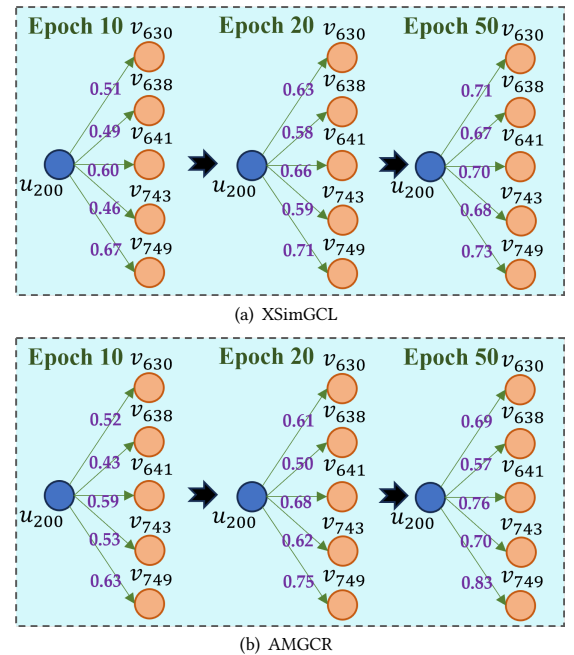


Figure 3: Case study of user preferences.

5.5 Case Study (RQ4)

In this section, we visualize the user preferences across training epochs on the Gowalla dataset to explore the model's ability of capturing user preferences. When the model converges, we compute the standard deviation (std) of every user preferences via Eq. 12 and then average them. We find that compared to DCCF (std=0.02) and XSimGCL (std=0.03), our AMGCR (std=0.06) yields larger discrepancy in preference, which suggests that our method is able

to effectively capture user preferences for diverse items. We also randomly sample an user (u_{200}) and visualize his (her) preferences for interacted items during the training process for XSimGCL and our AMGCR. The visualized results are presented in Figure 3 (a) and (b). For example, at epoch 50, user u_{200} 's preferences for different items appear to be close in XSimGCL. In contrast, our AMGCR shows the difference of user preferences. The underlying reason for these phenomena lies in the proposed preference refinement strategy, which enables the model to capture user preferences for different items during the learning process.

5.6 Hyper-parameter Analysis (RQ5)

In this section, we explore the sensitivity of our model with respect to four crucial hyper-parameters: λ_1 , λ_2 , d and B . The experimental results for loss weights of contrastive learning and preference refinement λ_1 and λ_2 are shown in Figure 4 (a) and (b). The experimental results for embedding size d and batchsize B shown in Table 5. We make the following observation.

❶ *Impact of λ_1 and λ_2 .* We find AMGCR reaches its best performance when increasing λ_1 to 0.2, and then it starts to deteriorate as becomes larger. This observation underscores the delicate balance required in setting to optimize model performance effectively, which also suggests that for contrastive learning as an auxiliary task, the weight of its loss should also be controlled within a reasonable range, so as not to exert excessive influence on the main task. ii) λ_2 has the same situation that the model gets the best performance when increasing to 0.5, and then it starts to deteriorate as becomes larger. iii) Consequently, we select the configuration with $\lambda_1 = 0.2$ and $\lambda_2 = 0.5$ as the optimal choice. In this configuration, the model strikes an effective balance between the main recommendation task, contrastive learning task, and preference refinement task, which facilitates a more comprehensive optimization process.

❷ *Impact of d and B .* Regarding the embedding size, we found that increasing it effectively enhances the model's performance. There is a clear trend where the Recall@20 improves as the embedding size increases. This indicates that the model is able to capture more useful information with larger embeddings, which is reflected in better performance. While there is a consistent increase, the rate of improvement decreases as the embedding size increases. On the other hand, the batchsize does not significantly impact the effectiveness of our model. The performance, as measured by Recall@20, remains relatively stable across different batch sizes. The maximum difference between the smallest and largest batch size is only 0.0014, indicating that increasing the batch size has a minimal impact on the model's effectiveness.

6 CONCLUSION

In this work, an effective GCL-based recommendation model is proposed, namely AMGCR, which employs adaptive augmentation to further improve the contrastive learning ability for recommendation. Four view generators are designed to learn the edge weights for constructing contrastive views, which alleviates the problem of noise introduction prevalent in current methods. In addition, the adaptive multi-view fusion module and preference refinement strategy are introduced to fuse the obtained four kinds of edge

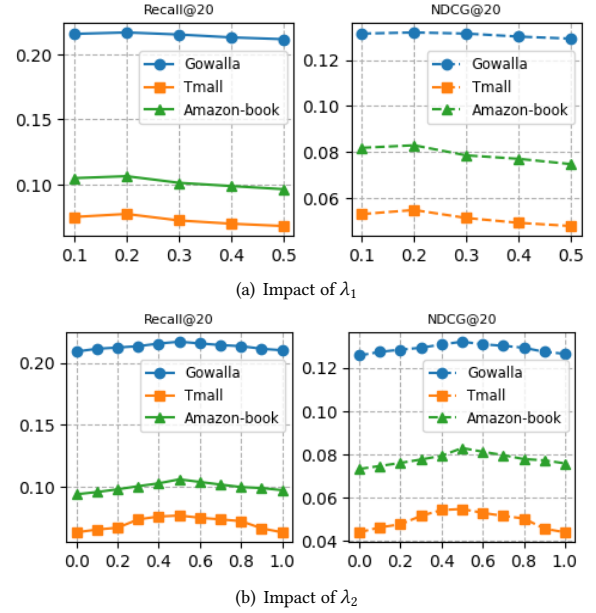


Figure 4: Influence of hyper-parameters.

Table 5: Influence of embedding size d and batchsize B on the Gowalla dataset.

Embedding size d	32	64	128	256
Recall@20	0.2170	0.2332	0.2544	0.2657
NDCG@20	0.1320	0.1369	0.1456	0.1526
Batchsize B	1024	2048	4096	10240
Recall@20	0.2159	0.2163	0.2170	0.2173
Recall@20	0.1309	0.1315	0.1320	0.1322

weights and empower the model to capture the preference information between users and items during the learning process, which further enhance the representation. Experimental results on three real-world datasets clearly validate the superiority of our model. In future work, we aim to optimize the model further, reducing its time complexity and enhancing its efficiency. In addition, we plan to delve deeper into refining learnable perspectives to advance GCL-based methods for superior recommendation performance.

ACKNOWLEDGMENTS

This work is supported by the National Key R&D Program of China 2023YFC3306303, the Key R&D Program of Zhejiang Province 2023C01213, and the Regional Innovation and Development Joint Fund of the National Natural Science Foundation of China (U22A6001). Jinshan Zhang was supported by the Key Research and Development Major Project of Zhejiang Province 2023C01002, Hangzhou Major Project and Development Program 2022AIZD0140 and Yongjiang Talent Introduction Program 2022A-236-G.

REFERENCES

- [1] Xuheng Cai, Chao Huang, Lianghao Xia, and Xubin Ren. 2023. LightGCL: Simple Yet Effective Graph Contrastive Learning for Recommendation. In *Proceedings of the Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda*. OpenReview.net.
- [2] Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. 2020. Revisiting Graph Based Collaborative Filtering: A Linear Residual Graph Convolutional Network Approach. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, New York, USA*. AAAI Press, 27–34.
- [3] Yongjun Chen, Zhiwei Liu, Jia Li, Julian J. McAuley, and Caiming Xiong. 2022. Intent Contrastive Learning for Sequential Recommendation. In *Proceedings of the ACM Web Conference, WWW 2022, Virtual Event, Lyon, France*. ACM, 2172–2182.
- [4] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhuan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, and Yong Li. 2021. Graph Neural Networks for Recommender Systems: Challenges, Methods, and Directions. *CoRR* abs/2109.12843 (2021).
- [5] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yong-Dong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China*. ACM, 639–648.
- [6] Xiangnan He, Zhankui He, Jingkuan Song, Zhenguang Liu, Yu-Gang Jiang, and Tat-Seng Chua. 2018. NAIS: Neural Attentive Item Similarity Model for Recommendation. *IEEE Trans. Knowl. Data Eng.* 30, 12 (2018), 2354–2366.
- [7] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web, WWW 2017, Perth, Australia*. ACM, 173–182.
- [8] Chao Huang, Huance Xu, Yong Xu, Peng Dai, Lianghao Xia, Mengyin Lu, Liefeng Bo, Hao Xing, Xiaoping Lai, and Yanfang Ye. 2021. Knowledge-aware Coupled Graph Neural Network for Social Recommendation. In *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Event*. AAAI Press, 4115–4122.
- [9] Yehuda Koren, Robert M. Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37.
- [10] Haoyang Li, Xin Wang, Ziwei Zhang, Zehuan Yuan, Hang Li, and Wenwu Zhu. 2021. Disentangled Contrastive Learning on Graphs. In *Proceedings of Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, virtual*. 21872–21884.
- [11] Zihan Lin, Changxin Tian, Yupeng Hou, and Wayne Xin Zhao. 2022. Improving Graph Collaborative Filtering with Neighborhood-enriched Contrastive Learning. In *Proceedings of the ACM Web Conference, WWW 2022, Virtual Event, Lyon, France*. ACM, 2320–2329.
- [12] Jianxin Ma, Peng Cui, Kun Kuang, Xin Wang, and Wenwu Zhu. 2019. Disentangled Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, Long Beach, California, USA, Vol. 97*. PMLR, 4212–4221.
- [13] Xubin Ren, Lianghao Xia, Jiashu Zhao, Dawei Yin, and Chao Huang. 2023. Disentangled Contrastive Collaborative Filtering. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan*. ACM, 1137–1146.
- [14] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. AutoRec: Autoencoders Meet Collaborative Filtering. In *Proceedings of the 24th International Conference on World Wide Web Companion, WWW 2015, Florence, Italy, Companion Volume*. ACM, 111–112.
- [15] Jianing Sun, Zhaoyue Cheng, Saba Zuberi, Felipe Pérez, and Maksims Volkovs. 2021. HGCF: Hyperbolic Graph Convolution Networks for Collaborative Filtering. In *Proceedings of the Web Conference, WWW 2021, Virtual Event / Ljubljana, Slovenia*. ACM / IW3C2, 593–601.
- [16] Rianne van den Berg, Thomas N. Kipf, and Max Welling. 2017. Graph Convolutional Matrix Completion. *CoRR* abs/1706.02263 (2017).
- [17] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *CoRR* abs/1807.03748 (2018).
- [18] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada*. OpenReview.net.
- [19] Chenyang Wang, Yuanqing Yu, Weizhi Ma, Min Zhang, Chong Chen, Yiqun Liu, and Shaoping Ma. 2022. Towards Representation Alignment and Uniformity in Collaborative Filtering. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*. ACM, 1816–1825.
- [20] Tongzhou Wang and Phillip Isola. 2020. Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event, Vol. 119*. PMLR, 9929–9939.
- [21] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural Graph Collaborative Filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2019, Paris, France*. ACM, 165–174.
- [22] Xiang Wang, Hongye Jin, An Zhang, Xiangnan He, Tong Xu, and Tat-Seng Chua. 2020. Disentangled Graph Collaborative Filtering. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China*. ACM, 1001–1010.
- [23] Xiao Wang, Ruijia Wang, Chuan Shi, Guojie Song, and Qingyong Li. 2020. Multi-Component Graph Convolutional Collaborative Filtering. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, New York, USA*. AAAI Press, 6267–6274.
- [24] Yifan Wang, Suyao Tang, Yuntong Lei, Weiping Song, Sheng Wang, and Ming Zhang. 2020. DisenHAN: Disentangled Heterogeneous Graph Attention Network for Recommendation. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management, CIKM 2020, Virtual Event, Ireland*. ACM, 1605–1614.
- [25] Wei Wei, Chao Huang, Lianghao Xia, Yong Xu, Jiashu Zhao, and Dawei Yin. 2022. Contrastive Meta Learning with Behavior Multiplicity for Recommendation. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining, WSDM 2022, Virtual Event / Tempe, AZ, USA*. ACM, 1120–1128.
- [26] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised Graph Learning for Recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, Canada*. ACM, 726–735.
- [27] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2023. Graph Neural Networks in Recommender Systems: A Survey. *ACM Comput. Surv.* 55, 5 (2023), 97:1–97:37.
- [28] Lianghao Xia, Chao Huang, Jiao Shi, and Yong Xu. 2023. Graph-less Collaborative Filtering. In *Proceedings of the ACM Web Conference, WWW 2023, Austin, TX, USA*. ACM, 17–27.
- [29] Lianghao Xia, Chao Huang, Yong Xu, Jiashu Zhao, Dawei Yin, and Jimmy X. Huang. 2022. Hypergraph Contrastive Collaborative Filtering. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain*. ACM, 70–79.
- [30] Lianghao Xia, Chao Huang, and Chuxu Zhang. 2022. Self-Supervised Hypergraph Transformer for Recommender Systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2022, Washington, DC, USA*. ACM, 2100–2109.
- [31] Tiansheng Yao, Xinyang Yi, Derek Zhiyuan Cheng, Felix X. Yu, Ting Chen, Aditya Krishna Menon, Lichan Hong, Ed H. Chi, Steve Tjoa, Jieqi (Jay) Kang, and Evan Ettinger. 2021. Self-supervised Learning for Large-scale Item Recommendations. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management, CIKM 2021, Virtual Event, Queensland, Australia*. ACM, 4321–4330.
- [32] Junliang Yu, Xin Xia, Tong Chen, Lizhen Cui, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2024. XSimGCL: Towards Extremely Simple Graph Contrastive Learning for Recommendation. *IEEE Trans. Knowl. Data Eng.* 36, 2 (2024), 913–926.
- [33] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. 2022. Are Graph Augmentations Necessary?: Simple Graph Contrastive Learning for Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain*. ACM, 1294–1303.
- [34] Jiani Zhang, Xingjian Shi, Shenglin Zhao, and Irwin King. 2019. STAR-GCN: Stacked and Reconstructed Graph Convolutional Networks for Recommender Systems. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China*. ijcai.org, 4264–4270.
- [35] Shengyu Zhang, Lingxiao Yang, Dong Yao, Yujie Lu, Fuli Feng, Zhou Zhao, Tat-Seng Chua, and Fei Wu. 2022. Re4: Learning to Re-contrast, Re-attend, Re-construct for Multi-interest Recommendation. In *Proceedings of the ACM Web Conference, WWW 2022, Virtual Event, Lyon, France*. ACM, 2216–2226.
- [36] Ding Zou, Wei Wei, Ziyang Wang, Xian-Ling Mao, Feida Zhu, Rui Fang, and Danyang Chen. 2022. Improving Knowledge-aware Recommendation with Multi-level Interactive Contrastive Learning. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM 2022, Atlanta, GA, USA*. ACM, 2817–2826.